

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

CAMPUS MARINGÁ – PARANÁ

JOÃO GABRIEL CARAÇATO PASTORELLI – RA: 25099878-2

JULIANO RAFAEL DE OLIVEIRA HAUARI – RA: 25291044-2

VITOR GABRIEL DA SILVA DUARTE – RA: 25141609-2

ECOLOG ALIMENTOS: SISTEMA LOGÍSTICO DE RESGATE E REDISTRIBUIÇÃO DE EXCEDENTES ALIMENTARES

Atividade de Estudo Programada (AEP) – 1º Bimestre / 4º Semestre (2026.2)

Repositório Oficial do Projeto no GitHub:

<https://github.com/judesune/EcoLog-Alimentos>

Maringá – PR

Setembro de 2026

1. Concepção, Descoberta e Alinhamento com os ODS

1.1. Contexto e Descoberta do Problema (Dores Identificadas)

Diariamente, toneladas de alimentos próprios para consumo humano são descartadas precocemente em supermercados, feiras e cooperativas agroindustriais unicamente por estarem próximas ao término de sua data de validade comercial. Em contrapartida, milhares de famílias em situação de vulnerabilidade social enfrentam insegurança alimentar crítica. Identificou-se que o obstáculo primário não é a indisponibilidade de excedentes doáveis, mas

sim um gargalo estritamente logístico e de gestão da informação: as instituições assistenciais não possuem visibilidade em tempo hábil sobre o que está disponível para resgate, nem capacidade de coordenar coletas rápidas antes que os produtos pereçam.

1.2. Alinhamento com a ODS da ONU

O projeto **EcoLog Alimentos** alinha-se diretamente ao **ODS 2 da ONU (Fome Zero e Agricultura Sustentável)**, com ênfase na **Meta 2.1**, que estabelece a necessidade de erradicar a fome e assegurar o acesso a alimentos seguros, nutritivos e suficientes durante todo o ano.

1.3. Proposta de Solução do Software

O **EcoLog Alimentos** atua como uma plataforma de retaguarda para organizar a cadeia de resgate. O sistema registra os pontos comerciais geradores de excedente, cadastra as entidades sociais que precisam dos donativos e controla as ordens de transporte logístico. O diferencial operacional reside no tratamento automatizado da prioridade de transporte: itens com validade mais curta e com exigência de cadeia fria (refrigeração) recebem prioridade máxima de despacho, garantindo que o alimento chegue à mesa de quem precisa antes de sua deterioração.

2. Escopo Funcional (Lista Sequencial de Requisitos)

Para solucionar o ciclo de coleta e redistribuição, os requisitos funcionais foram dispostos na ordem natural de execução do fluxo operacional:

1. **O sistema deve permitir o cadastro de pontos de coleta parceiros**, com razão social, CNPJ, endereço completo de coleta, telefone e e-mail de contato.
2. **O sistema deve permitir o cadastro de instituições receptoras de alimentos**, com razão social, CNPJ, capacidade de armazenamento (refrigerado e seco), endereço de entrega, telefone, e-mail e responsável institucional.
3. **O sistema deve permitir o cadastro de transportadores parceiros**, com razão social, CNPJ/CPF, tipo de veículo (caminhão, van ou furgão), capacidade volumétrica/peso de carga, motorista responsável, telefone e e-mail.
4. **O sistema deve permitir o registro de lotes de alimentos para resgate**, com descrição do produto, categoria, data de validade, quantidade em quilogramas e exigência de conservação térmica.
5. **O sistema deve permitir a criação de uma ordem de transporte logístico**, com código identificador, data agendada de coleta, ponto de coleta de origem, instituição de destino, status operacional inicial e a listagem de lotes de alimentos vinculados à carga.

6. **O sistema deve permitir a consulta e atualização do status de uma ordem de transporte**, com data da alteração, novo status (Pendente, Em Trânsito, Concluída ou Cancelada) e observações operacionais.
7. **O sistema deve permitir o cancelamento de uma ordem de transporte pendente**, com data do cancelamento e justificativa operacional detalhada do cancelamento.

3. Justificativa Técnica e Arquitetural

3.1. Linguagem de Programação: Java

- **Justificativa:** O Java foi escolhido por funcionar como uma ferramenta com "regras rígidas de conferência". Quando programamos em Java, o próprio computador aponta erros de digitação e inconsistências antes mesmo de o sistema rodar (tipagem estática).
- **Por que suporta o Escopo:** O sistema precisa lidar com diferentes tipos de comida ao mesmo tempo. Uma caixa de verduras estraga muito mais rápido que um saco de arroz e exige regras de transporte distintas. O Java permite criar um modelo geral de "Lote de Alimento" e programar variações ("Perecível" e "Não Perecível") que calculam automaticamente suas prioridades sem que precisemos reescrever código repetido, mantendo o software seguro e sem falhas de cadastro.

3.2. Banco de Dados: MySQL

- **Explicação Simplificada:** O MySQL funciona como uma grande planilha digital altamente organizada e protegida, em que as informações ficam gravadas em tabelas conectadas entre si por "chaves de segurança" (chaves estrangeiras / Foreign Keys).
- **Por que suporta o Escopo:** Em uma operação logística, nada pode ficar "perdido no ar". Se criarmos uma Ordem de Transporte, ela precisa estar obrigatoriamente amarrada a um ponto de coleta real e a uma instituição existente. O MySQL impede que uma entrega seja registrada sem destino ou que lotes de comida sumam do sistema. Se uma ordem inteira precisar ser excluída, o banco apaga automaticamente os itens vinculados a ela via cascata (ON DELETE CASCADE), garantindo que o banco nunca fique com dados desorganizados ou corrompidos.

3.3. Padrão Arquitetural: Camadas (Controller, Service, Repository, Model)

- **Explicação Simplificada:** Imagine um restaurante organizado: o **Garçom** anota o pedido (Controller), o **Gerente** confere se há estoque e se as regras foram seguidas

(Service), a **Despensa** guarda e busca os mantimentos (Repository), e as **Receitas/Ingredientes** são o produto em si (Model). Ninguém faz o trabalho do outro.

- **Por que suporta o Escopo:** Separar o sistema em camadas garante que, se no futuro quisermos mudar a forma de exibição (trocar a tela ou usar um aplicativo móvel), toda a lógica que calcula a data de validade das doações e a conexão com o banco continuarão intactas, sem necessidade de reconstruir o software do zero.

3.4. Ambiente e Sistemas Operacionais

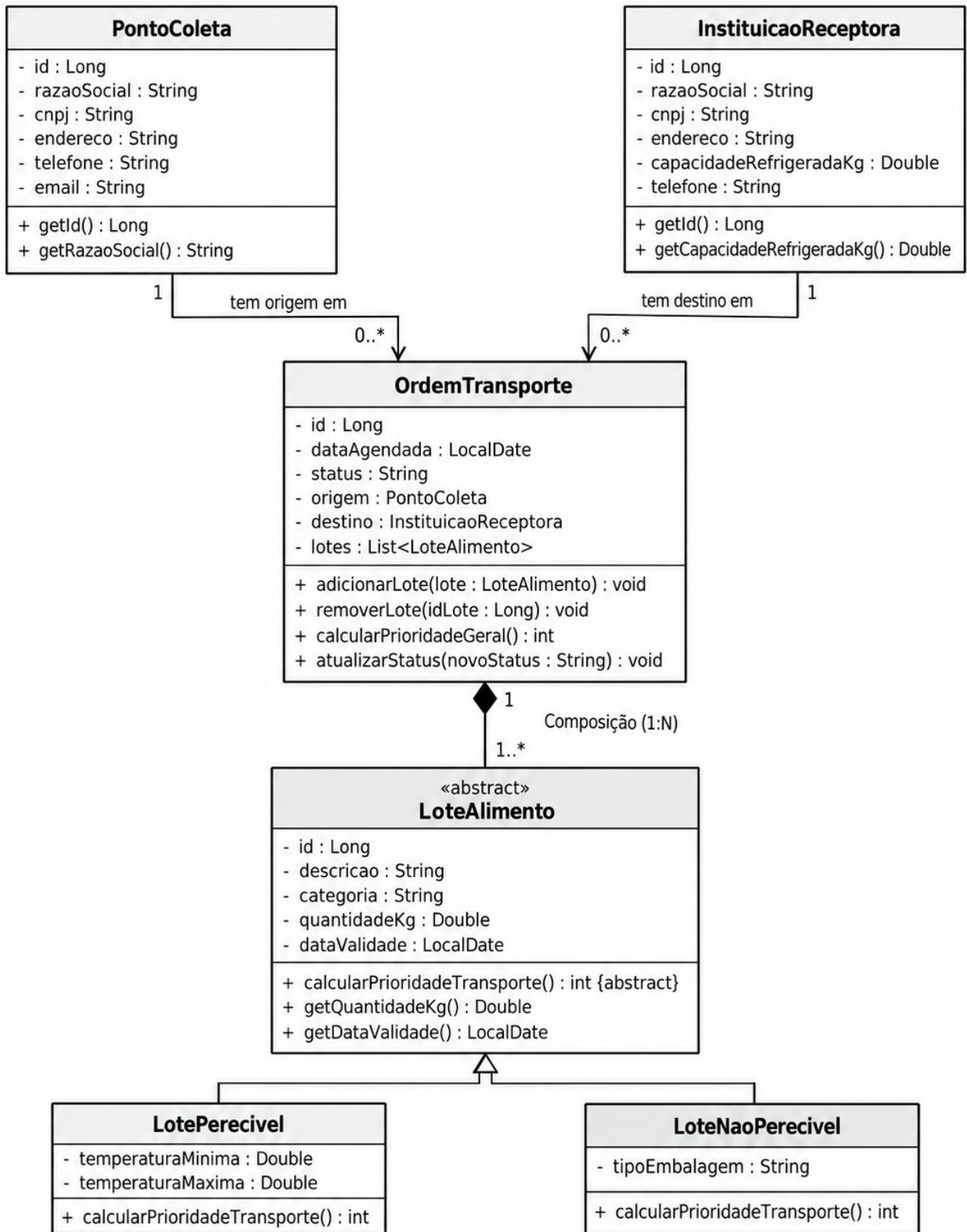
- **Multiplataforma:** A aplicação é compilada em Bytecode e executada pela Máquina Virtual Java (JVM), garantindo que funcione de forma idêntica em qualquer sistema operacional (Windows, Linux ou macOS) sem alterações de código-fonte.
- **Requisitos Mínimos:** Java Development Kit (JDK) versão 17 ou 21 instalada e servidor MySQL versão 8.0 ativo.

4. Modelagem do Sistema

4.1. Modelagem Orientada a Objetos (POO)

- **Entidade Principal para CRUD:** OrdemTransporte.
- **Composição com Multiplicidade 1:N:** A classe OrdemTransporte possui uma lista composta por 1 ou mais instâncias de LoteAlimento. Os lotes existem em função daquela remessa de transporte; cancelada a ordem antes do envio, a carga é desfeita.
- **Herança e Polimorfismo:**
 - **Superclasse Abstrata:** LoteAlimento, definindo atributos comuns (id, descricao, quantidadeKg, dataValidade) e o método abstrato:
Java
`public abstract int calcularPrioridadeTransporte();`
 - **Subclasses Especializadas:**
 - LotePercivel: Sobrescreve (@Override) o método calculando urgência alta com base nos dias restantes até o vencimento e na faixa de temperatura.
 - LoteNaoPercivel: Sobrescreve (@Override) o método com prioridade padrão orientada pelo volume e tipo de embalagem.

Diagrama de Classes



4.2. Modelagem do Banco de Dados (DER)

Tabelas relacionais projetadas para integridade referencial no MySQL:

1. tb_ponto_coleta: id_coleta (PK), razao_social, cnpj, logradouro, numero, telefone, email.
2. tb_instituicao_receptora: id_receptora (PK), razao_social, cnpj, logradouro, numero, capacidade_refrigerada_kg, telefone.
3. tb_ordem_transporte: id_ordem (PK), data_agendada, status, fk_id_coleta (FK), fk_id_receptora (FK).
4. tb_lote_alimento: id_lote (PK), descricao, categoria, quantidade_kg, data_validade, tipo_lote, temperatura_minima, temperatura_maxima, tipo_embalagem, fk_id_ordem (FK com ON DELETE CASCADE).

TB_PONTO_COLETA		
BIGINT	id_coleta	PK
VARCHAR(150)	razao_social	
VARCHAR(18)	cnpj	
VARCHAR(255)	endereco	
VARCHAR(20)	telefone	
VARCHAR(100)	email	

TB_INSTITUICAO_RECEPTORA		
BIGINT	id_receptora	PK
VARCHAR(150)	razao_social	
VARCHAR(18)	cnpj	
VARCHAR(255)	endereco	
DECIMAL(10_2)	capacidade_refrigerada_kg	
VARCHAR(20)	telefone	

origina

recebe

TB_ORDEM_TRANSPORTE		
BIGINT	id_ordem	PK
DATE	data_agendada	
VARCHAR(30)	status	
BIGINT	fk_id_coleta	FK
BIGINT	fk_id_receptora	FK

compos (1:N)

TB_LOTE_ALIMENTO		
BIGINT	id_lote	PK
VARCHAR(120)	descricao	
VARCHAR(50)	categoria	
DECIMAL(10_2)	quantidade_kg	
DATE	data_validade	
VARCHAR(20)	tipo_lote	
DECIMAL(5_2)	temperatura_minima	
DECIMAL(5_2)	temperatura_maxima	
VARCHAR(50)	tipo_embalagem	
BIGINT	fk_id_ordem	FK

5. Cronograma de Execução e Gestão de Prazos

Data Estimada	Atividade / Entrega	Etapa	Membro Responsável
06/09/2026	Definição da ODS 2.1, escopo e redação da Lista de Requisitos	1ª Entrega	Vitor
07/09/2026	Modelagem do Diagrama de Classes UML e DER do MySQL	1ª Entrega	João
10/09/2026	Justificativa técnica, organização do GitHub (/src, /docs, /database, README.md) e consolidação do PDF final	1ª Entrega	Juliano
15/10/2026	Criação do script de banco de dados (schema.sql em /database) e configuração da conexão JDBC	2ª Entrega	Juliano e João
25/10/2026	Implementação das classes de modelo com Herança, Polimorfismo e Composição	2ª Entrega	Vitor e João
25/10/2026	Implementação das operações completas do	2ª Entrega	Vitor e Juliano

	CRUD da entidade principal (OrdemTransporte)		
30/10/2026	Testes de execução no terminal (SO), homologação integrada e revisão final dos commits no repositório	2ª Entrega	Vitor, Juliano e João

FONTES

<https://brasil.un.org/pt-br/sdgs> - ODS da ONUS